# EECS 348 Assignment 4: Comparing Two AI-Generated Sudoku Solvers

**Francis Tryban · EECS 348 · October 15, 2026**

For this assignment, I gave two GenAIs the same C++ Sudoku problem and compared their programs. Their original code is included below. Claude Opus 5.5 helped me enter the prompts for the Codex run, make the test scripts, and put the results into an earlier draft. The comparison here comes from the programs and the saved test results in my working folder.

## 2a–2b. The two GenAIs

I used **Claude Haiku 4.5** in Claude Code through my Mac's terminal and **GPT-6 Luna** in the Codex desktop app on the same Mac. I gave each its own folder, and Codex was set to Light effort. I used these same two smaller models for Assignment 3 and wanted to see how they would handle a harder problem.

## 2c. The prompt

I put the same instructions in `Claude/PROMPT.md` and `Codex/PROMPT.md` and checked that the files matched exactly. I copied the program requirements from the assignment and added a short explanation of where to find the puzzle files. Each folder had the five puzzles and `solution1.txt`, and both `AGENT.md` files were empty. I told both models to read `AGENT.md` and `PROMPT.md`, write the C++ program, and save it as `iterations/00-raw.cpp`. Neither got the rubric or the other model's code.

Here is the full prompt both models received:

```
Write C++ code using objects, not functions, to implement the following requirements:

- Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.
- A typical puzzle is shown here (image credit: Wikimedia Commons): [image not reproduced]
- Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following
constraints:
  - Each digit from 1 to 9 must appear exactly once in each row.
  - Each digit from 1 to 9 must appear exactly once in each column.
  - Each digit from 1 to 9 must appear exactly once in each block.
- Published puzzles are usually constructed to have a unique solution. This requires that at least 17
cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as
78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are
already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.
- The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a)
solution; if no solution exists, prove that.
- It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle,
consider the bottom right block, top right cell. Because of other numbers in the same row, column, or
grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion,
using these and more advanced deduction rules, it's possible to gradually fill in the entire grid.
- That is not the approach we're going to use. Instead, we're going to implement a brute-force search,
in which we exhaustively try possibilities until we find a solution. This will be slower to run but is
easier to solve. (And by "slower to run," we mean it'll finish in a few tenths of a second rather than a
few milliseconds.)
```

```
- Here’s the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify
an empty cell and deduce a list of candidates for what might go into that cell. If there are no such
values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made
a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the
first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled
(i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and
so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates
report no solution, which again implies we took a wrong path somewhere earlier).
- This approach, by the way, is called depth-first search with backtracking. For each empty cell we make
an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find
something else to try, and so on. If a solution exists, we are guaranteed to find it, because we
exhaustively try all possibilities.
- You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits
already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
- Write a recursive object-oriented program to find the solution to the five puzzle files.
- The output for each solution, should include:
  - The puzzle file name (e.g., puzzle1.txt)
  - The puzzle from the file printed out.
  - The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.
- The solution may not be unique; if there is more than one solution to the puzzle, your program should
print them all out.
- To help you debug your program, solution1.txt is a solution to puzzle1.txt.

---

## Input clarification

The five puzzle files (puzzle1.txt through puzzle5.txt) and solution1.txt are
in the same folder as this file.

The program must find its puzzle files in either of two ways:

- If no command-line arguments are given, solve puzzle1.txt, puzzle2.txt,
  puzzle3.txt, puzzle4.txt and puzzle5.txt, in that order, from the current
  working directory: `./program`
- If one or more filenames are given as command-line arguments, solve those
  files instead, in the order given: `./program puzzle3.txt`

All output goes to standard output. The program exits with status 0 when it
completes normally.
```

# 2d. The programs they generated

## Claude Haiku 4.5 (`Claude/iterations/00-raw.cpp`)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class Sudoku {
private:
    std::vector<std::vector<int>> grid;
    static const int SIZE = 9;
    static const int BLOCK_SIZE = 3;

    bool isValid(int row, int col, int val) const {
        // Check row
        for (int c = 0; c < SIZE; c++) {
            if (grid[row][c] == val) return false;
        }

        // Check column
        for (int r = 0; r < SIZE; r++) {
            if (grid[r][col] == val) return false;
        }
```

```
        // Check 3x3 block
        int blockRow = (row / BLOCK_SIZE) * BLOCK_SIZE;
        int blockCol = (col / BLOCK_SIZE) * BLOCK_SIZE;
        for (int r = blockRow; r < blockRow + BLOCK_SIZE; r++) {
            for (int c = blockCol; c < blockCol + BLOCK_SIZE; c++) {
                if (grid[r][c] == val) return false;
            }
        }

        return true;
    }

    std::vector<int> getCandidates(int row, int col) const {
        std::vector<int> candidates;
        if (grid[row][col] != 0) return candidates;

        for (int val = 1; val <= SIZE; val++) {
            if (isValid(row, col, val)) {
                candidates.push_back(val);
            }
        }
        return candidates;
    }

    bool solve(bool& foundSolution) {
        // Find next empty cell
        for (int r = 0; r < SIZE; r++) {
            for (int c = 0; c < SIZE; c++) {
                if (grid[r][c] == 0) {
                    std::vector<int> candidates = getCandidates(r, c);

                    if (candidates.empty()) {
                        return false;
                    }

                    bool anyValid = false;
                    for (int val : candidates) {
                        grid[r][c] = val;

                        if (solve(foundSolution)) {
                            anyValid = true;
                        }

                        grid[r][c] = 0;
                    }
                    return anyValid;
                }
            }
        }

        // All cells filled - found a solution
        foundSolution = true;
        return true;
    }

public:
    Sudoku() : grid(SIZE, std::vector<int>(SIZE, 0)) {}

    bool loadFromFile(const std::string& filename) {
        std::ifstream file(filename);
        if (!file.is_open()) {
            return false;
        }

        for (int r = 0; r < SIZE; r++) {
            for (int c = 0; c < SIZE; c++) {
                char ch;
                file >> ch;
                if (ch == '_') {
                    grid[r][c] = 0;
                } else {
```

```
                    grid[r][c] = ch - '0';
                }
            }
        }

        file.close();
        return true;
    }

    void print() const {
        for (int r = 0; r < SIZE; r++) {
            if (r % BLOCK_SIZE == 0 && r != 0) {
                std::cout << "------+-------+------\n";
            }
            for (int c = 0; c < SIZE; c++) {
                if (c % BLOCK_SIZE == 0 && c != 0) {
                    std::cout << "| ";
                }
                if (grid[r][c] == 0) {
                    std::cout << "_ ";
                } else {
                    std::cout << grid[r][c] << " ";
                }
            }
            std::cout << "\n";
        }
    }

    Sudoku copy() const {
        Sudoku newPuzzle;
        newPuzzle.grid = this->grid;
        return newPuzzle;
    }

    bool solve() {
        bool foundSolution = false;
        solve(foundSolution);
        return foundSolution;
    }
};

int main(int argc, char* argv[]) {
    std::vector<std::string> files;

    if (argc == 1) {
        files = {"puzzle1.txt", "puzzle2.txt", "puzzle3.txt", "puzzle4.txt", "puzzle5.txt"};
    } else {
        for (int i = 1; i < argc; i++) {
            files.push_back(argv[i]);
        }
    }

    for (const auto& filename : files) {
        std::cout << "Puzzle: " << filename << "\n";

        Sudoku puzzle;
        if (!puzzle.loadFromFile(filename)) {
            std::cout << "Could not open file: " << filename << "\n\n";
            continue;
        }

        std::cout << "Original:\n";
        puzzle.print();
        std::cout << "\n";

        if (puzzle.solve()) {
            std::cout << "Solution:\n";
            puzzle.print();
        } else {
            std::cout << "No solution found\n";
        }
```

```
        std::cout << "\n";
    }

    return 0;
}
```

### GPT-6 Luna (`Codex/iterations/00-raw.cpp`)

```
#include <array>
#include <fstream>
#include <iostream>
#include <string>
#include <vector>

class Sudoku {
private:
    std::array<std::array<int, 9>, 9> cells{};
    std::vector<std::array<std::array<int, 9>, 9>> solutions;
    static constexpr std::size_t solutionLimit = 100000;

    bool allowed(int row, int column, int value) const {
        for (int i = 0; i < 9; ++i) {
            if (cells[row][i] == value || cells[i][column] == value)
                return false;
        }

        int blockRow = (row / 3) * 3;
        int blockColumn = (column / 3) * 3;
        for (int r = blockRow; r < blockRow + 3; ++r) {
            for (int c = blockColumn; c < blockColumn + 3; ++c) {
                if (cells[r][c] == value)
                    return false;
            }
        }
        return true;
    }

    void search() {
        if (solutions.size() >= solutionLimit)
            return;

        int row = -1;
        int column = -1;
        for (int r = 0; r < 9 && row == -1; ++r) {
            for (int c = 0; c < 9; ++c) {
                if (cells[r][c] == 0) {
                    row = r;
                    column = c;
                    break;
                }
            }
        }

        if (row == -1) {
            solutions.push_back(cells);
            return;
        }

        for (int value = 1; value <= 9; ++value) {
            if (allowed(row, column, value)) {
                cells[row][column] = value;
                search();
                cells[row][column] = 0;
                if (solutions.size() >= solutionLimit)
                    return;
            }
        }
    }
```

```
public:
    bool load(const std::string& filename) {
        std::ifstream input(filename);
        if (!input)
            return false;

        std::string line;
        int row = 0;
        while (row < 9 && std::getline(input, line)) {
            int column = 0;
            for (char ch : line) {
                if (ch == ' ' || ch == '\t' || ch == '\r' || ch == '|')
                    continue;
                if (ch >= '1' && ch <= '9') {
                    cells[row][column++] = ch - '0';
                } else if (ch == '_') {
                    cells[row][column++] = 0;
                }
                if (column == 9)
                    break;
            }
            if (column == 9)
                ++row;
            else if (column != 0)
                return false;
        }
        if (row != 9)
            return false;

        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                int value = cells[r][c];
                if (value != 0) {
                    cells[r][c] = 0;
                    bool valid = allowed(r, c, value);
                    cells[r][c] = value;
                    if (!valid)
                        return true; // Search will correctly report no solutions.
                }
            }
        }
        return true;
    }

    void solve() {
        solutions.clear();
        search();
    }

    void printPuzzle() const {
        printGrid(cells);
    }

    void printSolutions() const {
        if (solutions.empty()) {
            std::cout << "No solution found\n";
            return;
        }
        for (std::size_t i = 0; i < solutions.size(); ++i) {
            if (solutions.size() > 1)
                std::cout << "Solution " << i + 1 << ":\n";
            printGrid(solutions[i]);
        }
    }

private:
    static void printGrid(const std::array<std::array<int, 9>, 9>& grid) {
        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                if (c != 0)
```

```
                    std::cout << ' ';
                std::cout << (grid[r][c] == 0 ? '_' : char('0' + grid[r][c]));
            }
            std::cout << '\n';
        }
    }
};

class SudokuProgram {
private:
    std::vector<std::string> filenames;

public:
    SudokuProgram(int argc, char* argv[]) {
        if (argc > 1) {
            for (int i = 1; i < argc; ++i)
                filenames.emplace_back(argv[i]);
        } else {
            for (int i = 1; i <= 5; ++i)
                filenames.push_back("puzzle" + std::to_string(i) + ".txt");
        }
    }

    int run() const {
        for (std::size_t i = 0; i < filenames.size(); ++i) {
            const std::string& filename = filenames[i];
            if (i != 0)
                std::cout << '\n';
            std::cout << filename << '\n';

            Sudoku puzzle;
            if (!puzzle.load(filename)) {
                std::cout << "Unable to read puzzle\n";
                continue;
            }
            puzzle.printPuzzle();
            puzzle.solve();
            puzzle.printSolutions();
        }
        return 0;
    }
};

int main(int argc, char* argv[]) {
    SudokuProgram program(argc, argv);
    return program.run();
}
```

## 2e. My comparison

At first, the programs looked very similar to me. Both find the first empty square, try the digits 1 to 9 that fit, and back up when a choice does not work. That is the backtracking method the assignment asks for. Claude put the puzzle work in one `Sudoku` class and handled the files in `main`. Codex used a second class, `SudokuProgram`, for the files, leaving only 4 lines in `main`. I expected them to give similar answers because they search the same way. (Edit: I was wrong)

### Correctness

Both programs compiled without errors or warnings, including with the extra warning flags. Then I ran the five course puzzles. I checked the printed grids against a separate Python program that counted the correct solutions, and I checked puzzle 1 against `solution1.txt`.

| Puzzle | Correct answer | Claude | Codex |
|---|---|---|---|
| puzzle1 | 1 solution | Printed the unsolved puzzle | Correct, matches `solution1.txt` |
| puzzle2 | 1 solution | Printed the unsolved puzzle | Correct |
| puzzle3 | 1 solution | Printed the unsolved puzzle | Correct |
| puzzle4 | No solution | “No solution found” | “No solution found” |
| puzzle5 | 2 solutions | Printed the unsolved puzzle | Printed both solutions |

Claude’s search reaches the answers, but it erases each number again as it backs up. It never saves a finished grid. By the time the program prints “Solution:”, it is printing the original puzzle again. There is another problem with puzzle 5: the output part of Claude’s program only prints one grid, even though that puzzle has 2 answers. Claude’s own report claimed it printed “only the first solution,” but the tests show that it did not print a solution at all.

I wanted to know whether this was just a problem with the course puzzles, so I asked AI. A third AI (Opus 5.5 High Effort) tried 34 random puzzles, one with 196 solutions, one with 29,040 solutions, and a hard puzzle. Codex matched the separate solution counter every time. Claude did not produce the right solutions for any of those tests. Both could read the course files with Windows line endings and copies saved with Mac/Linux line endings.

I also checked what happened with missing or unusual files:

| Test | Claude | Codex |
|---|---|---|
| Missing file | Says it could not open the file | Says “Unable to read puzzle” |
| File with 8 rows, a letter, or nothing in it | Treats it as a puzzle and prints junk numbers like 72 and -48 | Says “Unable to read puzzle” |
| Completely blank grid (billions of solutions) | Still running after 10 seconds | Stops after 100,000 solutions without saying it stopped |

The assignment says the puzzle files will have the right format, so the bad-file tests are extra. The blank grid exposed a problem that does matter to the instructions. Codex stops after 100,000 solutions without telling the user. If a puzzle has more answers than that, it has not really printed “all” the solutions.

### Execution time

The more blank squares there are, the more choices the programs may have to try. Each blank can have up to 9 possible digits, so the worst-case number of paths grows roughly like **O(9^b)**, where **b** is the number of blanks. Most paths end early because they break a Sudoku rule. Both programs try the same squares and digits in the same order, so neither has a smarter search. On the hard puzzle, each made 2,068,781 calls to its search method.

### Space complexity

Both start with one 9x9 grid. Each time they fill a blank, they go one step deeper, so even a completely empty grid cannot take them past 82 levels. Those steps use 128 bytes each for Claude and 80 bytes each for Codex. For a fixed-size Sudoku puzzle, that working space does not keep growing without a limit.

Codex also keeps a copy of every answer until it is ready to print them. Each copy takes 324 bytes. That means its storage grows with the number of answers, or **O(s)** space for **s** solutions. These measurements (conducted by my same Opus 5.5 on High) show it: on the five course puzzles, both programs peaked at 2,816 KB. On the puzzle with 29,040 solutions, Claude still used 2,816 KB and Codex used 13,044 KB. On the blank grid, Codex reached 44,248 KB at its 100,000-solution limit. Claude uses less memory because it does not keep the finished answers, which is also why its output is wrong.

### Maintainability

I also thought about which program I could read and fix more easily. Neither has the required opening comment with my name, a description, inputs, outputs, and sources. That is to be expected, it was not in the prompt. Claude has 5 comment lines and Codex has 1.

Claude's short methods were easier for me to follow at first. It has a `copy()` method, but never calls it when it finds an answer. Codex separates file handling, solving, and printing more clearly, and its `main` is only 4 lines. Its hardest part to follow is `load`: it does a lot of input checking in one method. On the lecture's measure of how many decisions are packed into a method, `load` scored 20, above the suggested limit of 15. Claude's highest score was 8. ## 2f. Which program I chose

I chose Codex's program. The biggest reason is that it was right. Codex got all 5 course puzzles right and matched the solution counter on every other test. Claude only got puzzle 4 right, the one with no answer, because it never saves the finished grid. Speed was about the same for both, so that did not decide it. Codex used more memory when a puzzle had a lot of answers, but I could fix that without changing how it searches.

I also liked how Codex was built. In *A Philosophy of Software Design*, Ousterhout says a good module is "deep": simple to use on the outside, with the hard work hidden inside. Codex's `SudokuProgram` is like that. `main` just makes it and calls `run()`. Claude put most of the work in a long `main`, and its solving and saving are mixed together, which is right where its bug is. Fixing Claude would mean rewriting the main part of the program. Codex's one weak spot was `load`, which did too much in one function, so that was the first thing I fixed.

## 2g. How I improved it

Claude Opus 5.5 helped me write 2f and 2g, and ChatGPT (GPT-6.1 Sol) helped me revise the code and comments. Both are listed in my prolog. Here is what I changed from Codex's original:

- **Print each answer right away.** Codex saved every answer in a list before printing. Now each one prints as soon as it is found, so memory stays about the same no matter how many answers there are.
- **Removed the hidden 100,000 limit.** Codex quietly stopped after 100,000 answers. Now it really prints all of them, like the assignment asks.
- **Split up `load`.** It had a complexity of 20, over the limit of 15. I broke it into `load`, `readGrid`, `readRow` and `checkClues`, and moved "find the first empty cell" into its own function, `findEmpty`. The most complex function now scores 7.
- **Skip the search when two clues already break a rule**, since there can't be an answer.
- **Better names.** The grid type is now just `Grid`, and new variables have names that say what they hold, like `cluesClash` and `solutionCount`.
- **Comments and prolog.** Codex had 1 comment line and no prolog. I rewrote the comments so every line is explained (185 comment lines) and added the full prolog.
- **Output.** Every answer is numbered, and each puzzle ends with "Solutions found: N" like `solution1.txt`.

Before and after (same tests, same machine):

| | Codex original | My final version |
|---|---:|---:|
| Course puzzles right | 5 of 5 | 5 of 5 |
| Blank grid | stops at 100,000 without saying so | prints every answer |
| Memory, 29,040-answer puzzle | 13,836 KB | 3,752 KB |
| Memory, blank grid | 45,036 KB | 3,456 KB |
| Time, 29,040-answer puzzle | 0.99 s | 0.97 s |
| Time, hard puzzle | 1.48 s | 1.44 s |
| Most complex function | 20 | 7 |
| Comment lines | 1 | 185 |

The time barely changed because the search does the same work. The real wins are memory, actually printing every answer, and code that is easier to read. I built the final program on cycle1 with `g++ Assignment4.cpp -o Assignment4.exe`, and it printed the right answer for all five puzzles.